- abstract class: cannot be instantiated.
- abstract method: subclasses must implement it.
- keywords: abstract, interface, implements, @Override
- Interface: a contract the classes implement.

Two common problems:

- leaky abstraction: other code depends on details that were never declared as part of the abstraction.
- Over-abstraction: too much is hidden, so users lack control or access.

Adherence criteria:

- Meaningful labels: nouns for classes and variables, verbs for methods.
- Context-aware labels: no redundancy with the class name.
- Explicit declaration of the full abstraction.
- Sufficiency: users can do everything they need.
- Concisenss: no redundant, testability, maintainability, and reusability.

Users Interface Design

Good UI design starts with understanding people (their goals, tasks, skills, and attitudes), not with pictures.

Design methodologies:

- Human-centered: follows human physiological facts.
- User-centered: driven by an accurate understanding of users (persons, profiles, scenarios).
- Usage-centered: based on usage and tasks (use cases, role, tak and content models).
- Ineffective approaches: driven by database, language, or platform.

Raskin's 4 laws of a humane interface:

1. Don't harm the user's work.
2. Don't waste their time.
3. Be responsive to human needs and frailties.
4. Let the user set the pace

Complexity: essential complexity (inherent to the problem) vs. accidental complexity (added by us). Aim to minimize the accidental kind.

Why people use software: find something, learn, perform transactions, control or monitor, create, communicate, be entertained, make decisions.

User behavior patterns (Tidwell): safe exploration, instant gratification, satisficing, changes in midstream, deferred choices, incremental construction, habituation, microbreaks, spatial memory, prospection memory, streamlined repetition, keyboard only, other people's advice, perso

DD MM AA

Polymorphism

Geek for "many forms": related objects are treated as the same general type, and each one does the correct task at runtime. It is one of the three pillars of OO, with encapsulation and inheritance.

Problem without it: client code full of `if/else` + `instanceof` checks. It works, but it is verbose, error-prone, and hard to maintain whe a new subclass is added. Fix in 3 steps:

1. Define a common interface in a abstract superclass (e.g. `getPossibleMoves()`).
2. Each subclass implements it with `@Override`.
3. The client depends only on the superclass and just calls the method.

The client code is shorter and cleaner, but what it does is only know at runtime.

New behavior goes into new subclasses and the client code stays unchanged (Extensibility). A new definition of an inherited method in a subclass.

Overloading: several methods with the same name but different parameters in the same class.

Example:

```
for(Shape s: shapes) { s.draw(); } // each Shape subclass
```

Abstraction

- Abstraction (verb): considering only the relevant details of an object and hiding the rest.
- Abstraction (noun): a description, such as a class definition, that leaves out unnecessary details.

Relevance depends on context. A chiken class for a farm system differs from one for a coop simulator. Public members are the exposed details. Private members and implementation details are hidden. A more abstract class has fewer details (ChessPiece is more abstract than Pawn).

- Technique: find the similarities between systems, put them in an abstract layer, and leave the differences to concrete subclasses. The abstract layer cannot stand alone.

Java tools

nal recommendations.

- Metaphor: a familiar ideal desktop, control panel, shopping cart that guides design and narrows the gap between user and software the, a recognizable interface style with its own vocabulary, such as form, text editors, spreadsheets, browsers, media players, or games.

- UI challenge: it takes more than programming knowledge of users, standard widgets, and visual composition. It keeps getting harder because of more deveces, mor complex apps, higgher exceptions and more diverse users.